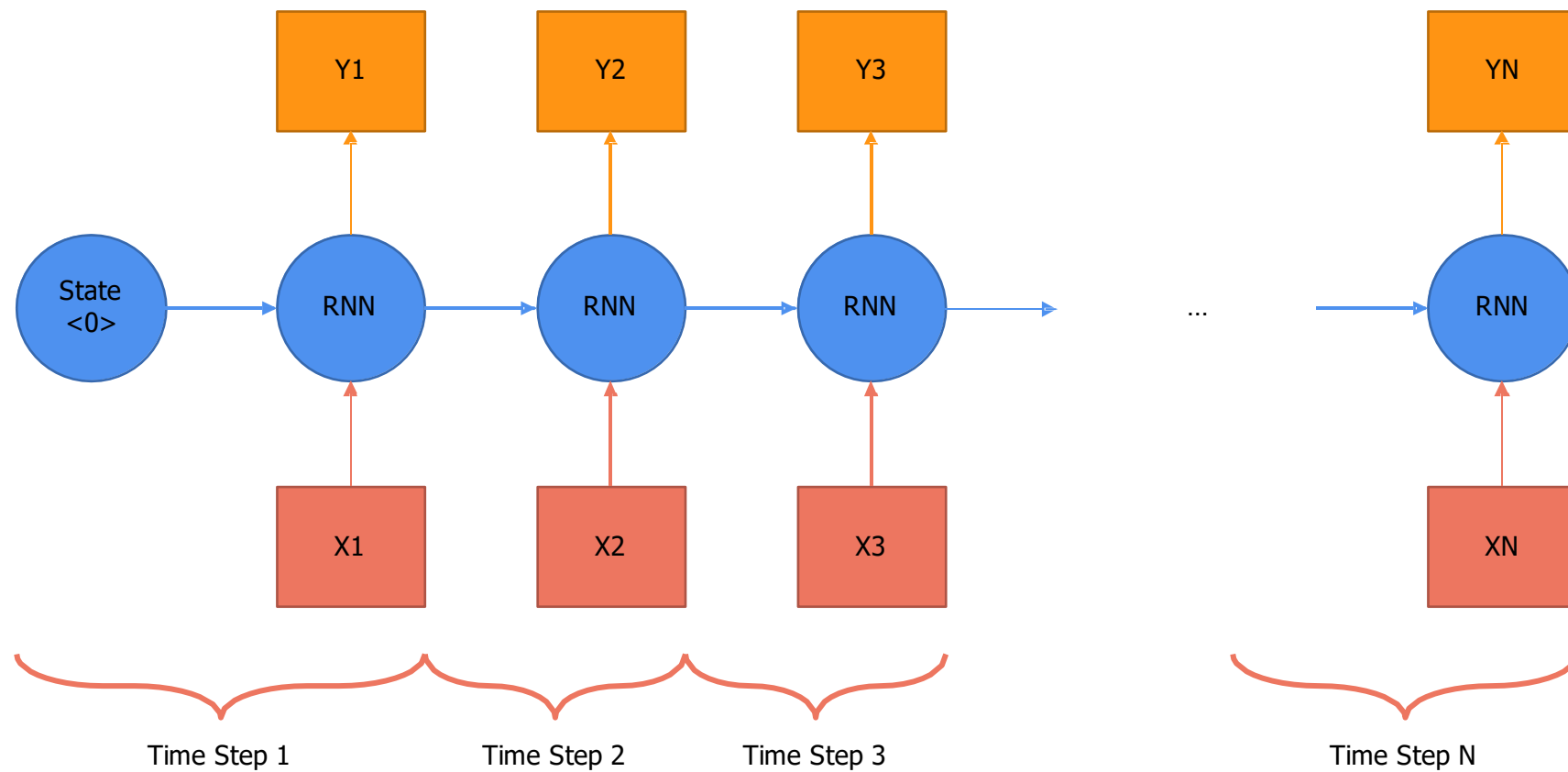


Transformer

(Attention is All You Need)

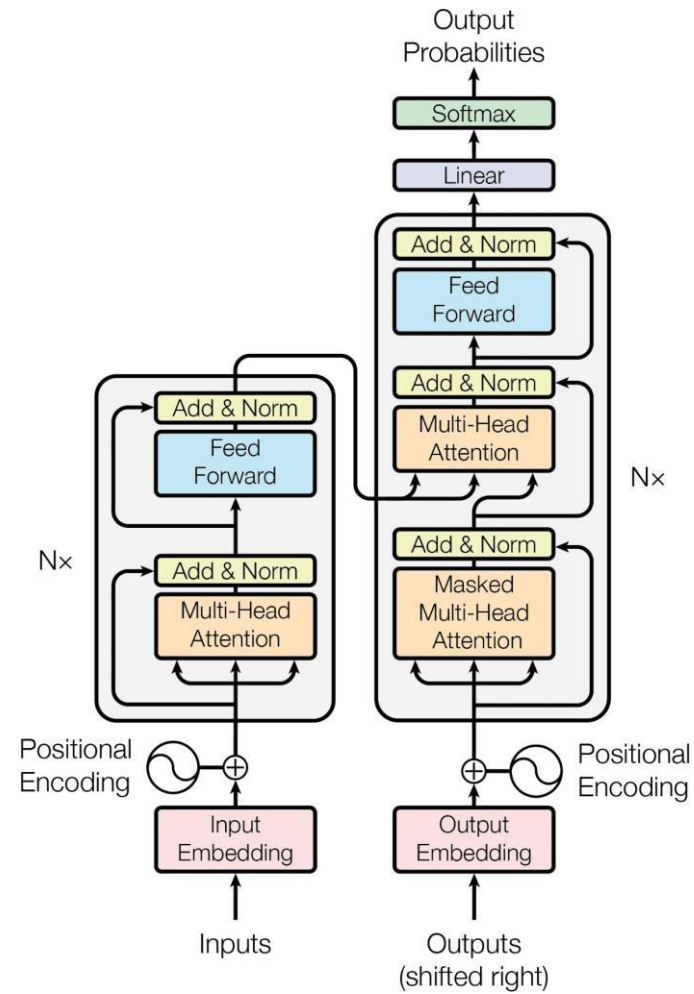
Recurrent Neural Networks (RNN)



Problems with RNN (among others)

1. Slow computation for long sequences
2. Vanishing or exploding gradients
3. Difficulty in accessing information from long time ago

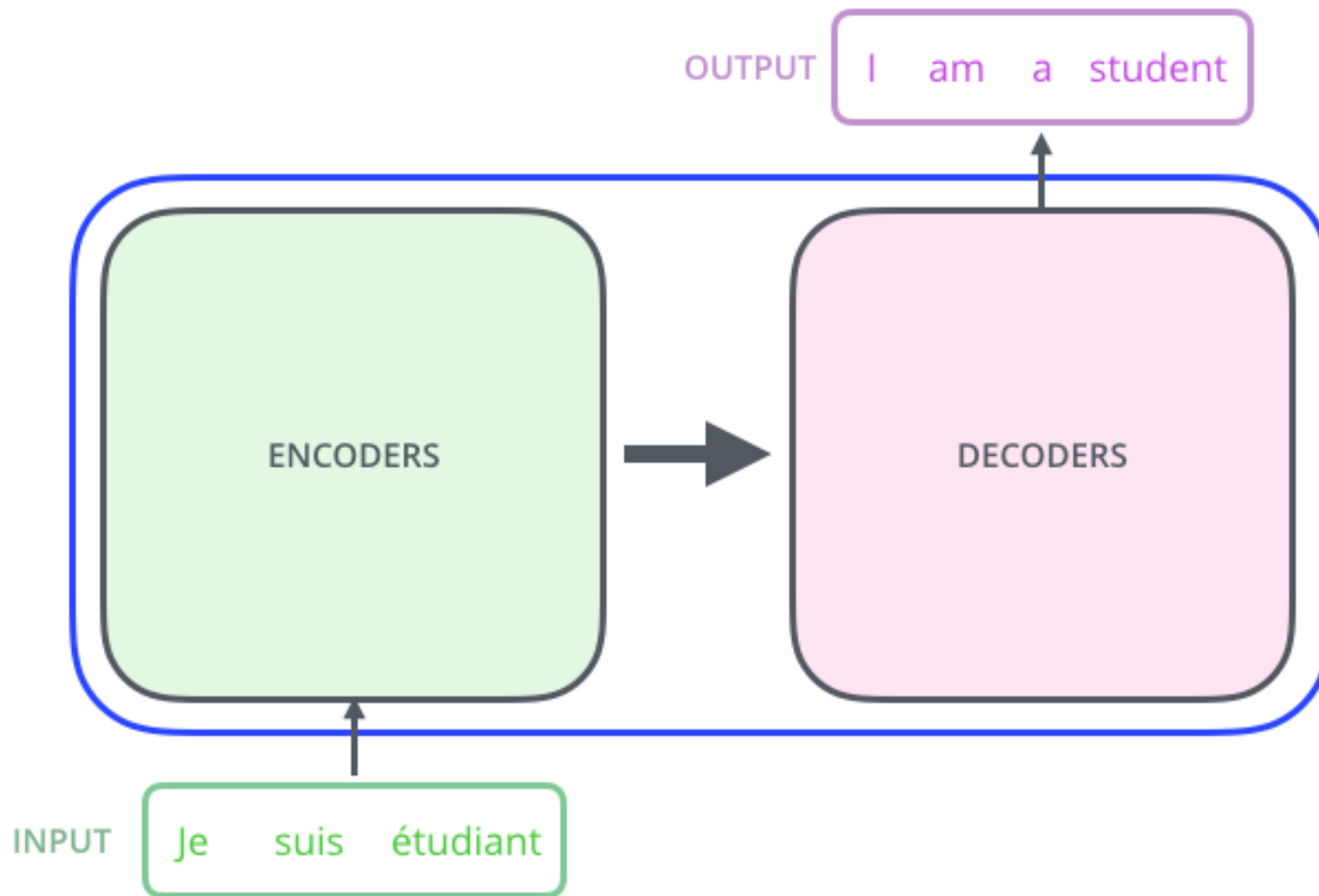
Transformers

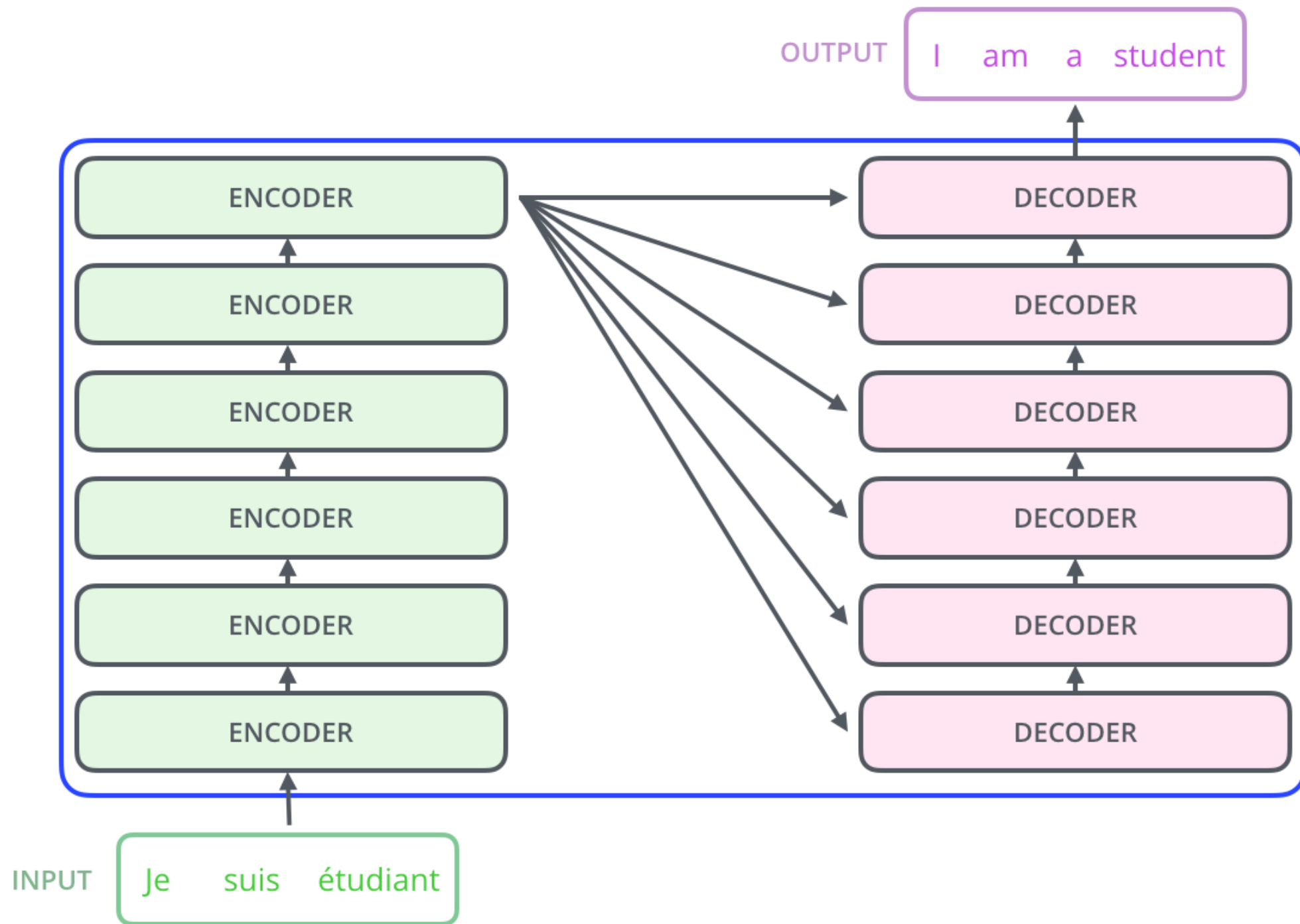


Transformer (A High-Level Look)

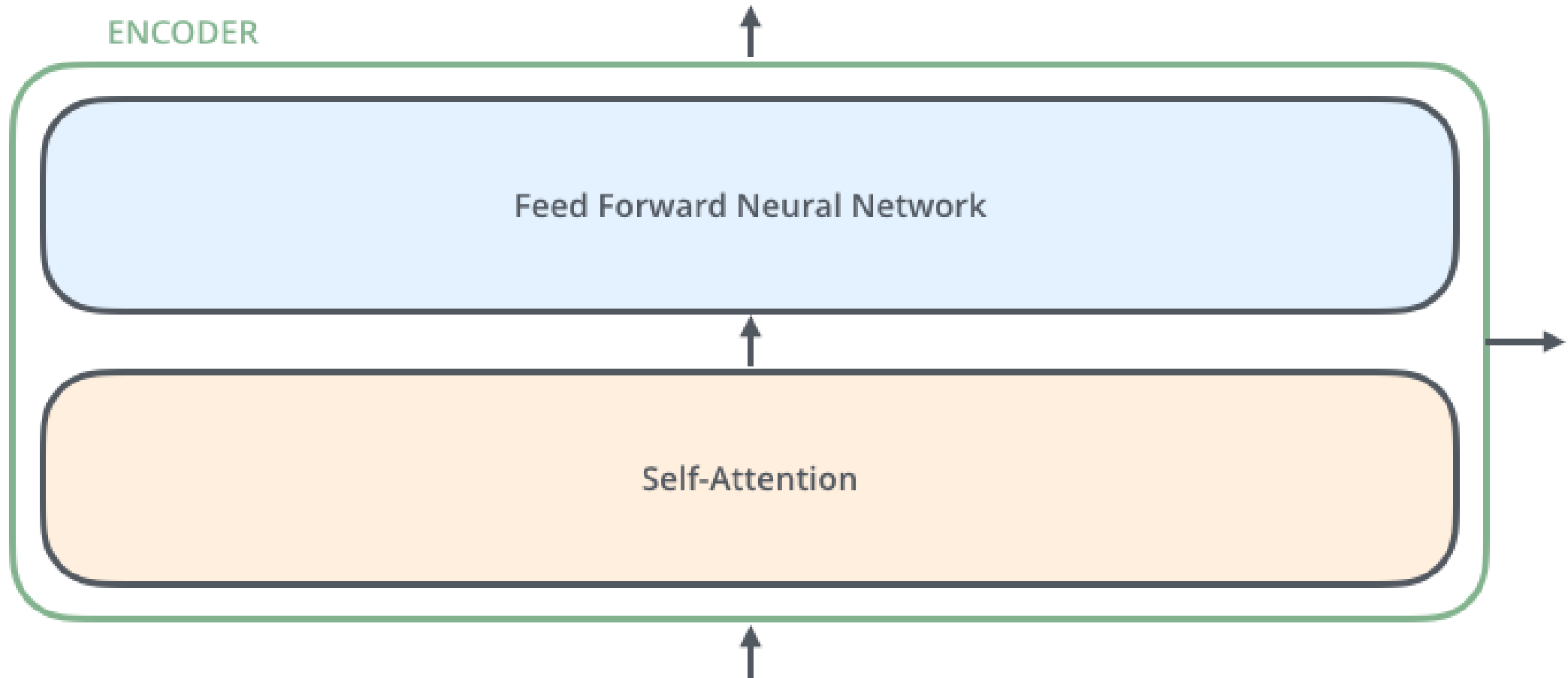


Transformer (A High-Level Look)

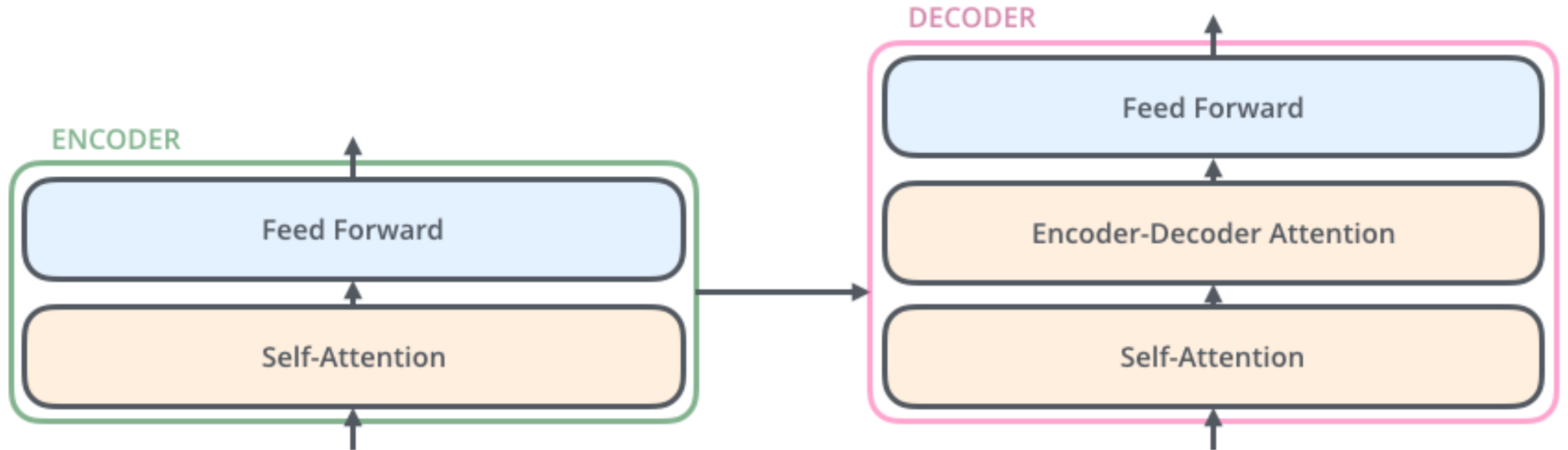




The encoders are all identical in structure. Each one is broken down into two sub-layers:



The encoders are all identical in structure. Each one is broken down into two sub-layers:



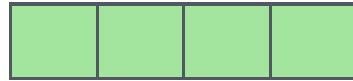
Each word is embedded into a vector of size 512. We'll represent those vectors with these simple boxes.

x_1



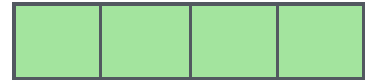
Je

x_2



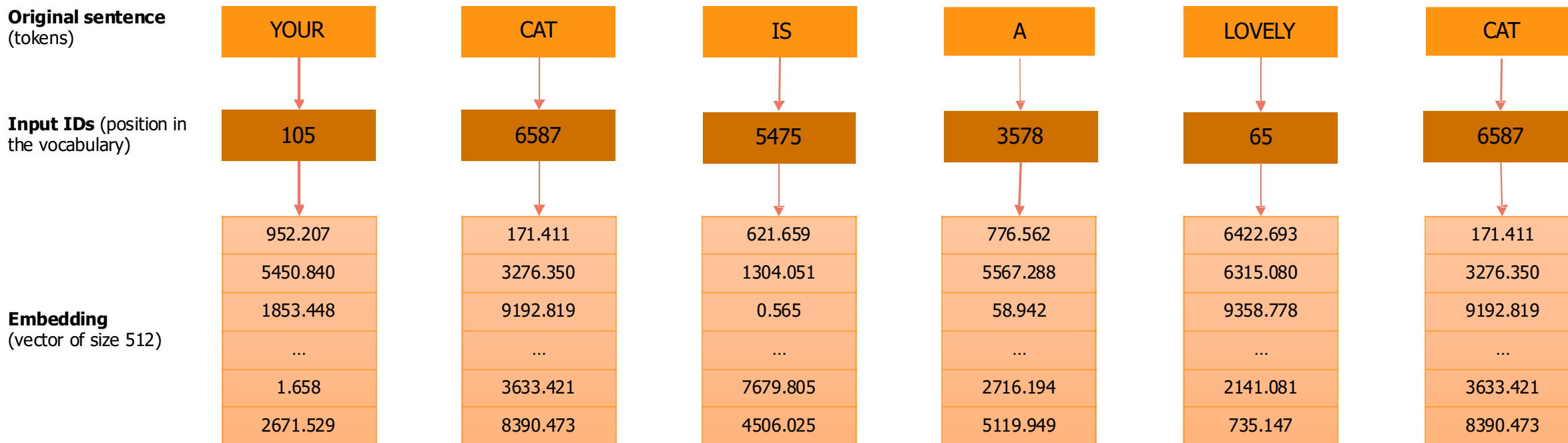
suis

x_3

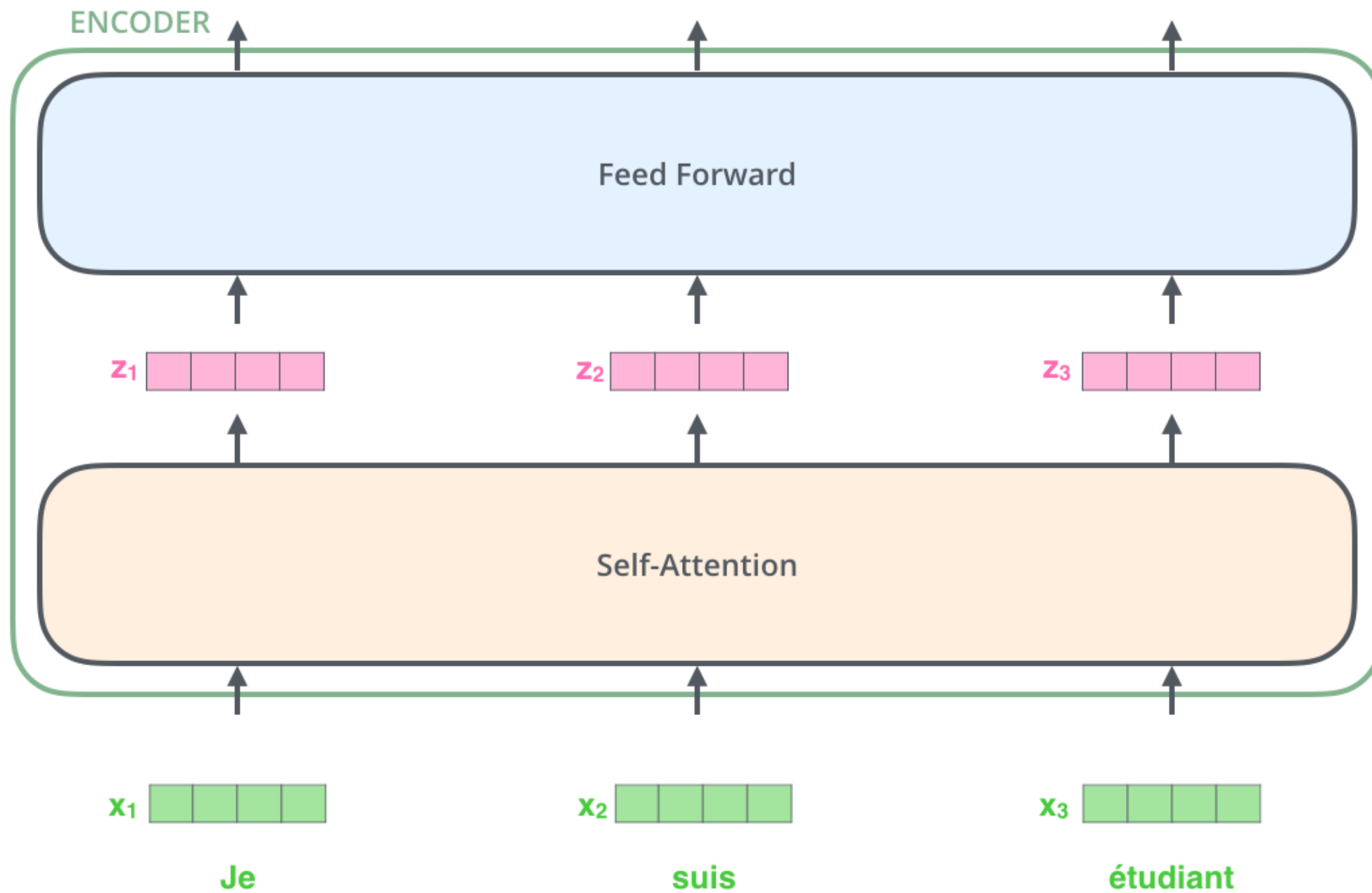


étudiant

What is an input embedding?

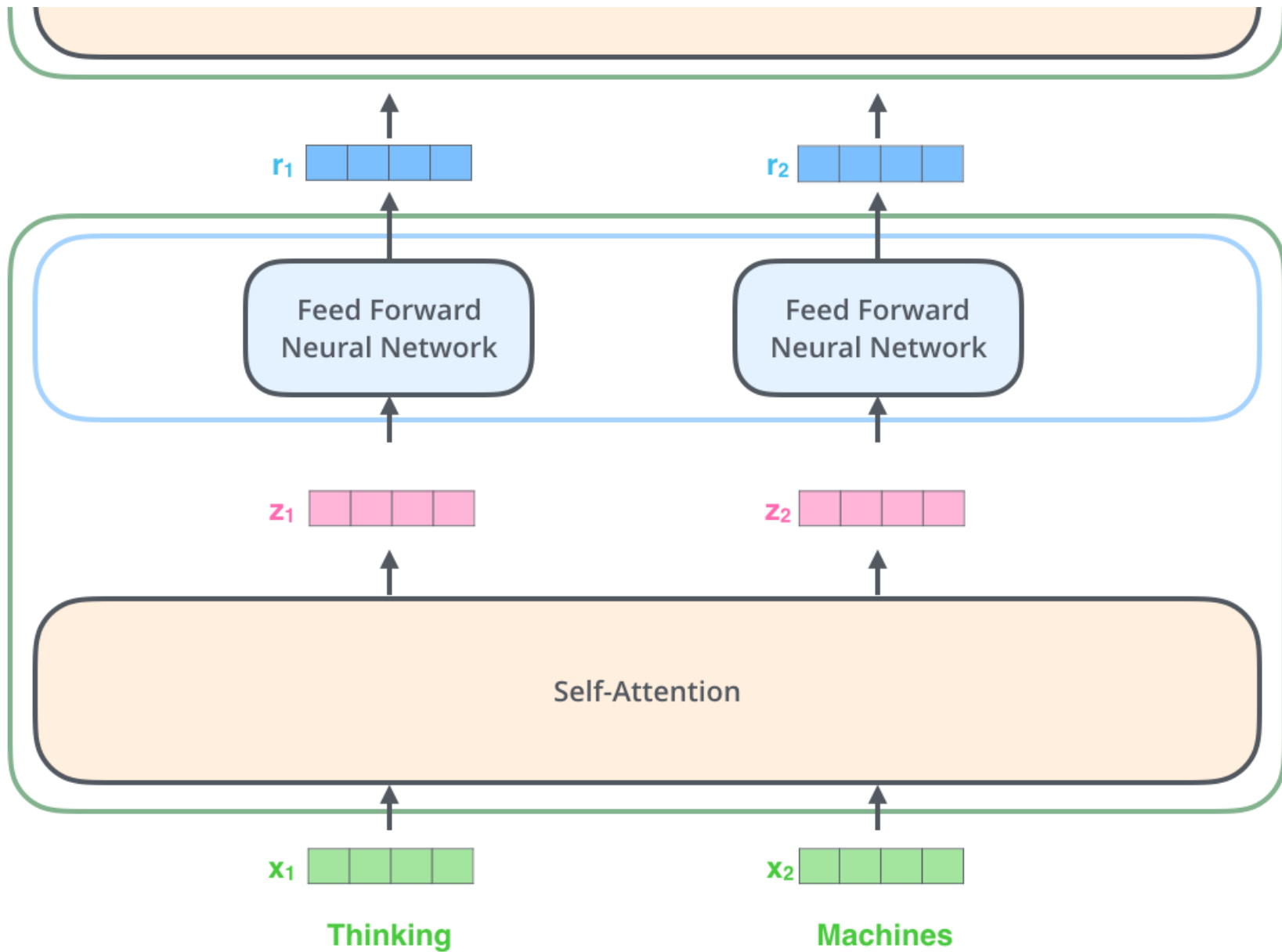


We define $d_{\text{model}} = 512$, which represents the size of the embedding vector of each word

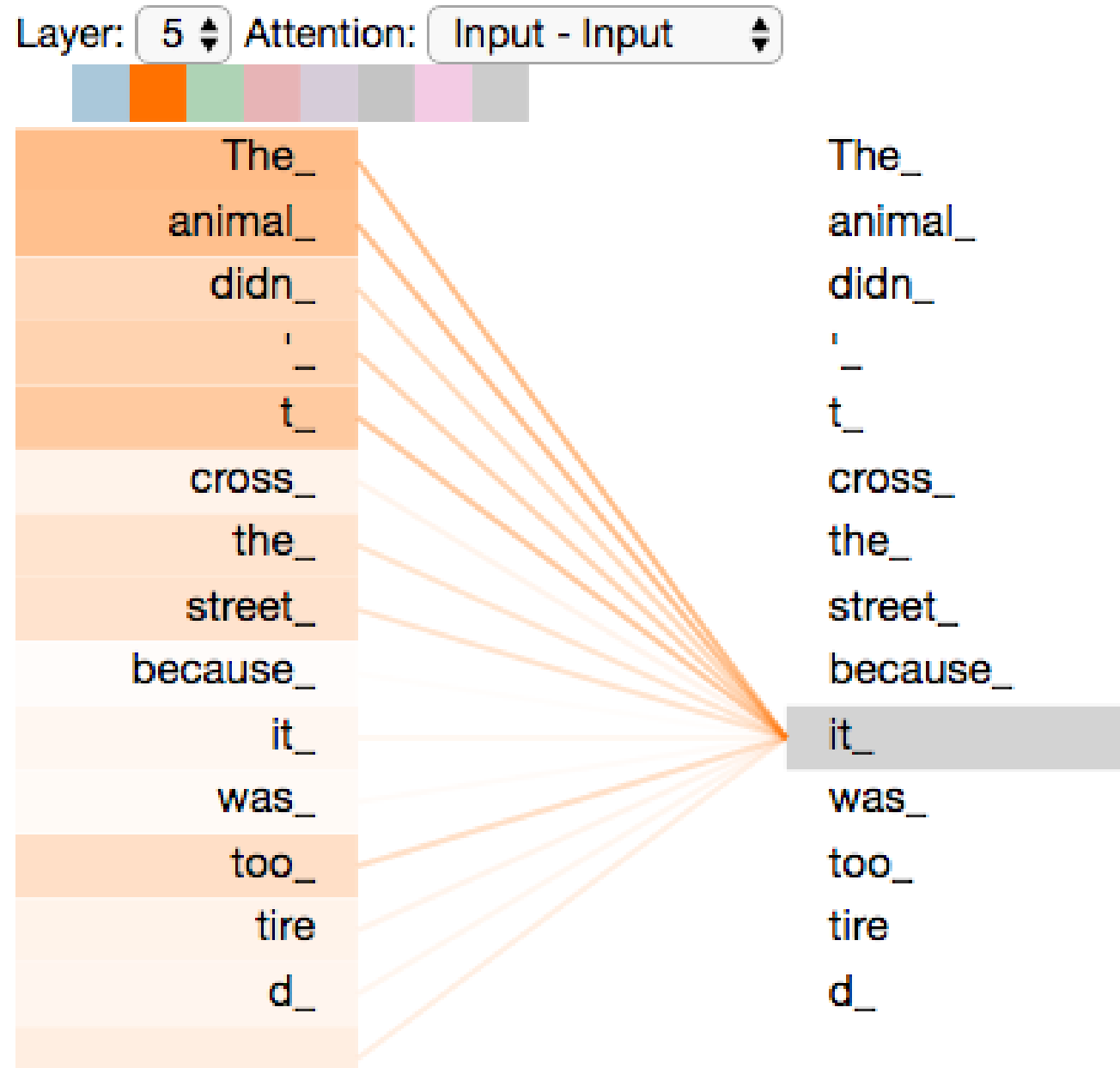


ENCODER #2

ENCODER #1



The animal didn't cross the street because it was too tired”

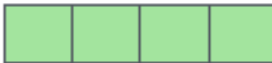


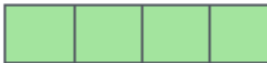
Input

Thinking

Machines

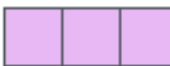
Embedding

x_1 

x_2 

Queries

q_1 

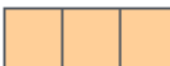
q_2 



W^Q

Keys

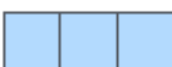
k_1 

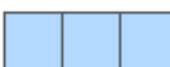
k_2 



W^K

Values

v_1 

v_2 



W^V

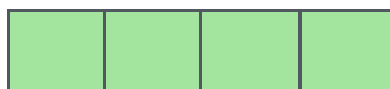
Input

Thinking

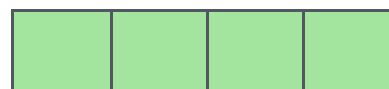
Machines

Embedding

x_1



x_2

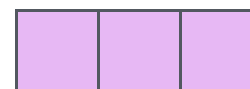


Queries

q_1



q_2



Keys

k_1



k_2



Values

v_1



v_2



Score

$$q_1 \cdot k_1 = 112$$

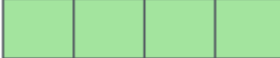
$$q_1 \cdot k_2 = 96$$

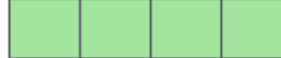
Input

Thinking

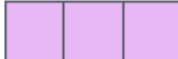
Machines

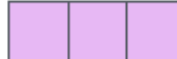
Embedding

x_1 

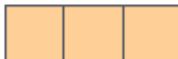
x_2 

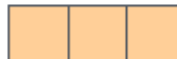
Queries

q_1 

q_2 

Keys

k_1 

k_2 

Values

v_1 

v_2 

Score

$$q_1 \cdot k_1 = 112$$

$$q_1 \cdot k_2 = 96$$

Divide by 8 ($\sqrt{d_k}$)

14

12

Softmax

0.88

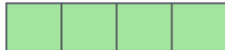
0.12

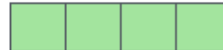
Input

Thinking

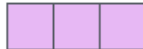
Machines

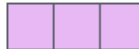
Embedding

x_1 

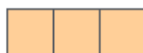
x_2 

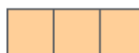
Queries

q_1 

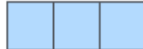
q_2 

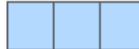
Keys

k_1 

k_2 

Values

v_1 

v_2 

Score

$q_1 \cdot k_1 = 112$

$q_1 \cdot k_2 = 96$

Divide by 8 ($\sqrt{d_k}$)

14

12

Softmax

0.88

0.12

Softmax

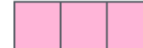
X

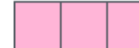
Value

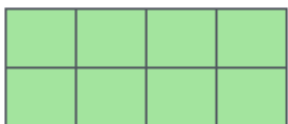
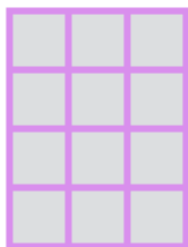
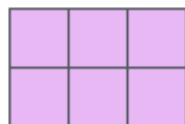
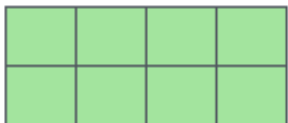
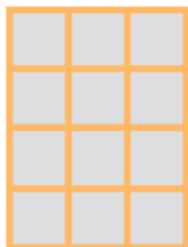
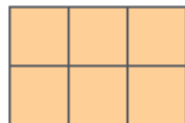
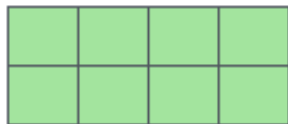
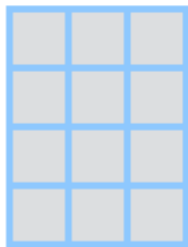
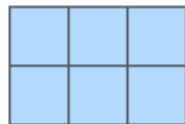
v_1 

v_2 

Sum

z_1 

z_2 

X  $\times$ W^Q  $=$ Q  X  $\times$ W^K  $=$ K  X  $\times$ W^V  $=$ V 

The self-attention calculation in matrix form

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

=

Z

$\begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}$

Practice Question

Given the sentence: " smart robot quickly" you need to compute the encoder self-attention output for the word "robot" using the provided weight matrices for query, key, and value and the embeddings of input words (x_{smart} is embedding for the word "smart").

$$x_{\text{smart}} = \begin{bmatrix} 2 \\ 1.5 \end{bmatrix}, \quad x_{\text{robot}} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}, \quad x_{\text{quickly}} = \begin{bmatrix} 0.8 \\ 0 \end{bmatrix},$$

$$W_Q = \begin{bmatrix} 0 & 0.7 \\ 2.5 & 1.6 \end{bmatrix}, \quad W_K = \begin{bmatrix} 0.8 & 0.7 \\ 0 & 0.5 \end{bmatrix}, \quad W_V = \begin{bmatrix} 2 & 0.5 \\ 1.5 & 1 \end{bmatrix}$$

Solution

$$X = \begin{bmatrix} 2 & 1.5 \\ 1.5 & 0.5 \\ 0.8 & 0 \end{bmatrix} \quad (\text{smart, robot, quickly})$$

$$Q = XW_Q = \begin{bmatrix} 3.75 & 3.8 \\ 1.25 & 1.85 \\ 0 & 0.56 \end{bmatrix}$$

$$K = XW_K = \begin{bmatrix} 1.6 & 2.15 \\ 1.2 & 1.3 \\ 0.64 & 0.56 \end{bmatrix}$$

$$V = XW_V = \begin{bmatrix} 6.25 & 2.5 \\ 3.75 & 1.25 \\ 1.6 & 0.4 \end{bmatrix}$$

$$Q_{robot} = [1.25 \quad 1.85]$$

$$\text{scores} = Q_{robot}K^T = [5.9775, 3.905, 1.836]$$

$$\frac{\text{scores}}{\sqrt{2}} = [4.23, 2.76, 1.30]$$

$$\alpha = \text{softmax} = [0.776, 0.178, 0.046]$$

Final attention output

$$= [0.776, 0.178, 0.046] \begin{bmatrix} 6.25 & 2.5 \\ 3.75 & 1.25 \\ 1.6 & 0.4 \end{bmatrix} = [5.60, 2.08]$$

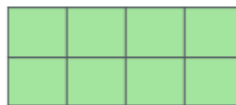
The Beast With Many Heads

Multi Head Attention

- It expands the model's ability to focus on different positions
- It gives the attention layer multiple “representation subspaces”

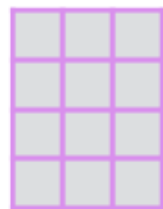
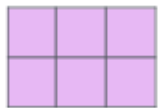
X

Thinking
Machines



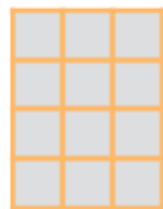
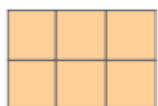
ATTENTION HEAD #0

Q₀



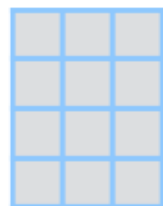
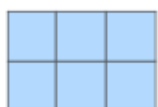
W₀^Q

K₀



W₀^K

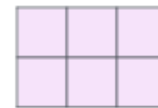
V₀



W₀^V

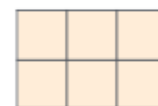
ATTENTION HEAD #1

Q₁



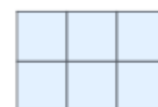
W₁^Q

K₁



W₁^K

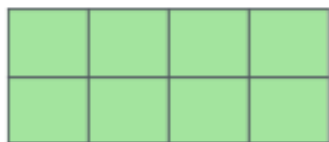
V₁



W₁^V

X

Thinking
Machines

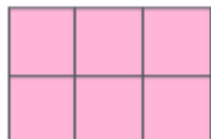


Calculating attention separately in
eight different attention heads



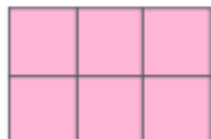
ATTENTION
HEAD #0

Z_0



ATTENTION
HEAD #1

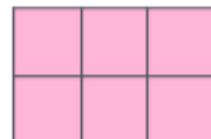
Z_1



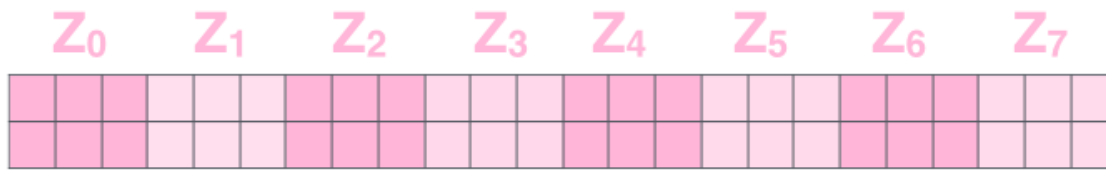
...

ATTENTION
HEAD #7

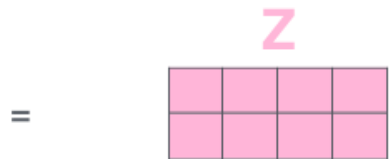
Z_7



1) Concatenate all the attention heads

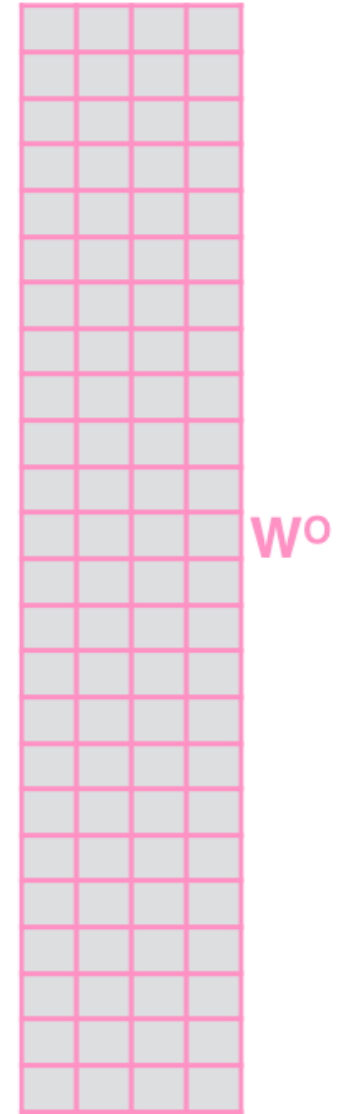


3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



2) Multiply with a weight matrix W^O that was trained jointly with the model

X



1) This is our input sentence*

2) We embed each word*

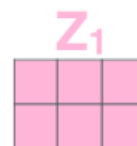
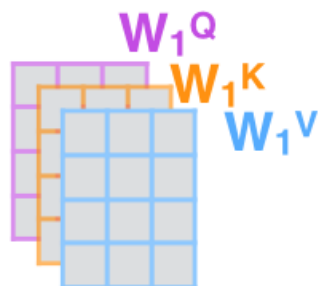
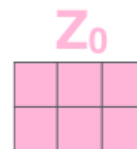
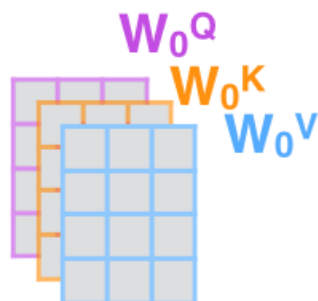
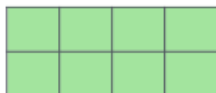
3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking
Machines

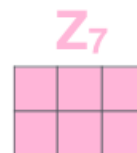
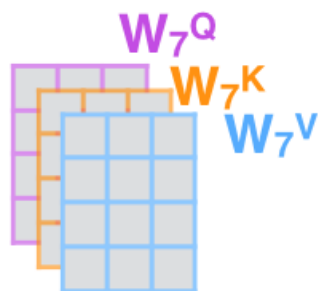
X



...

...

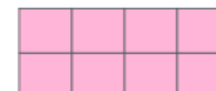
...



W^O

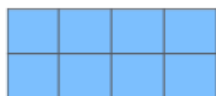


Z

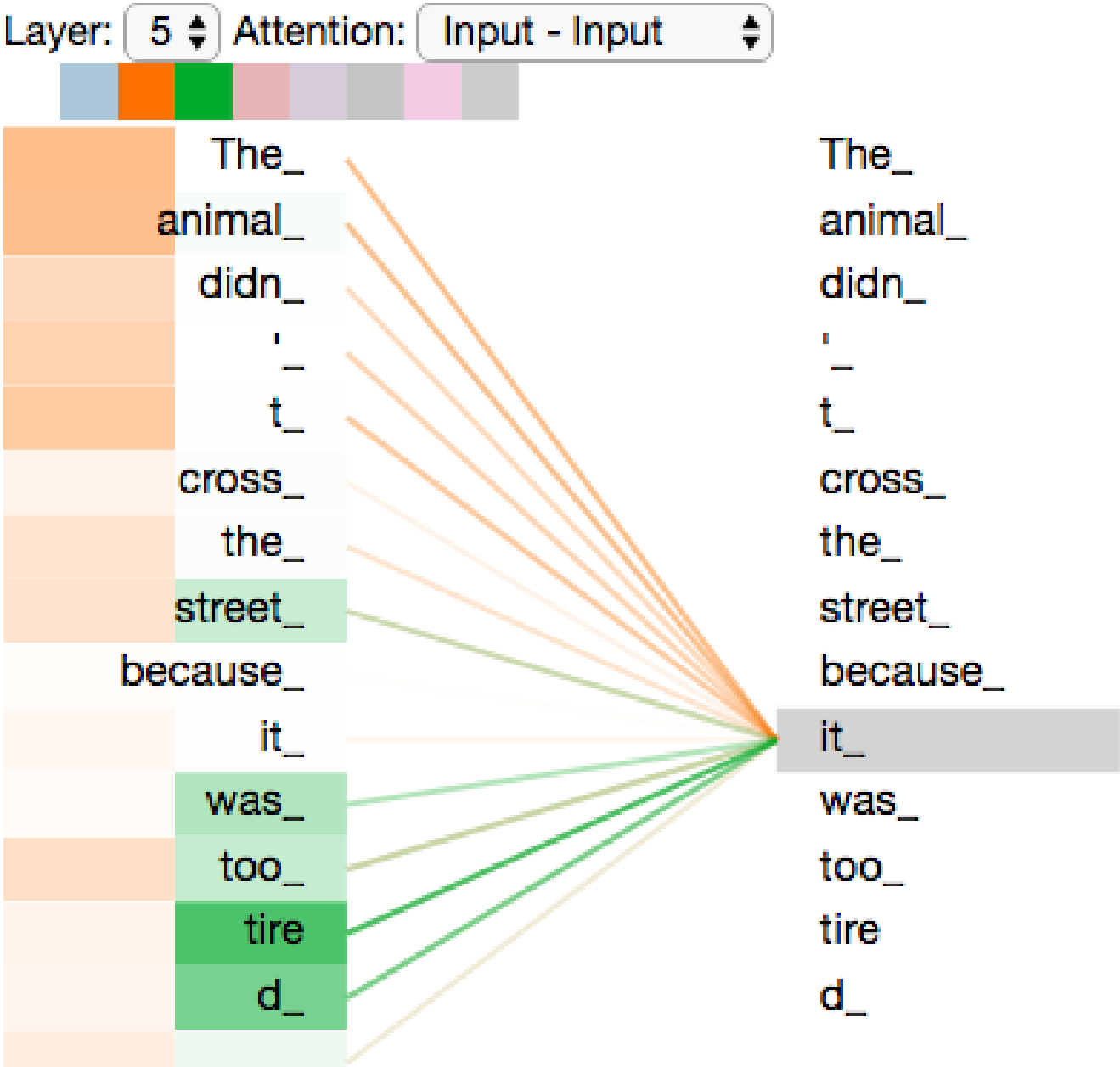


* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

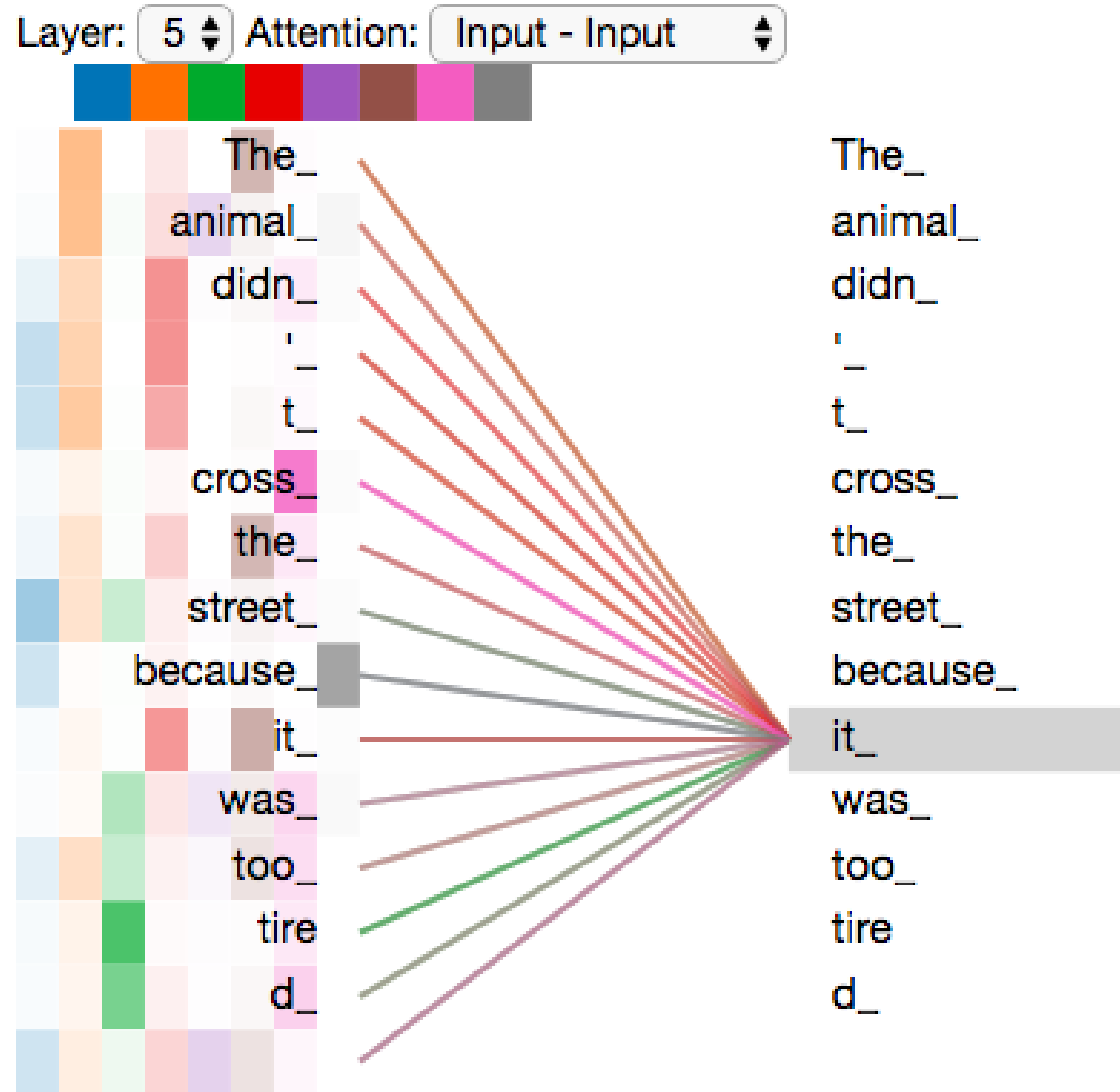
R



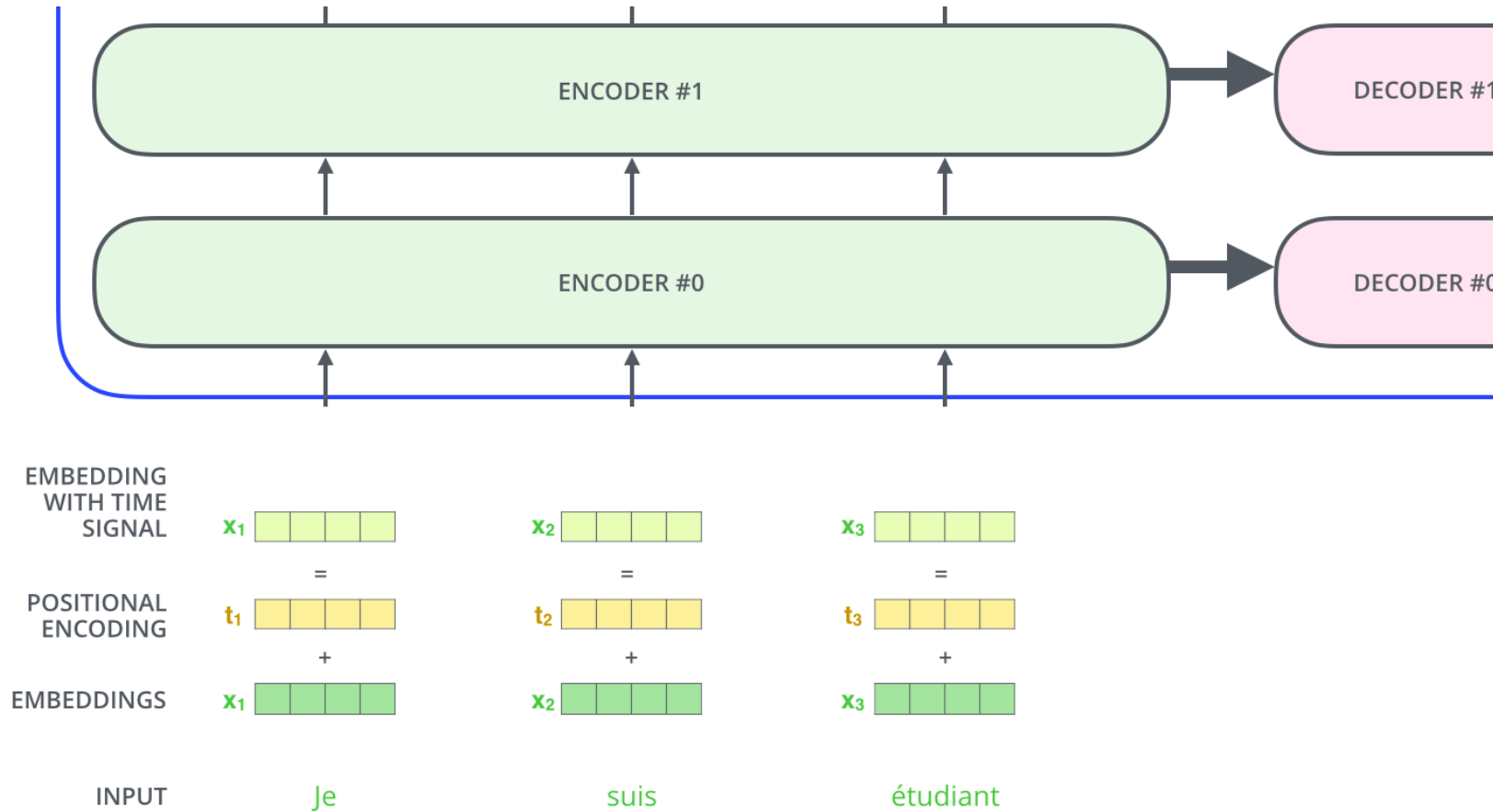
Multi Head Attention



Multi Head Attention



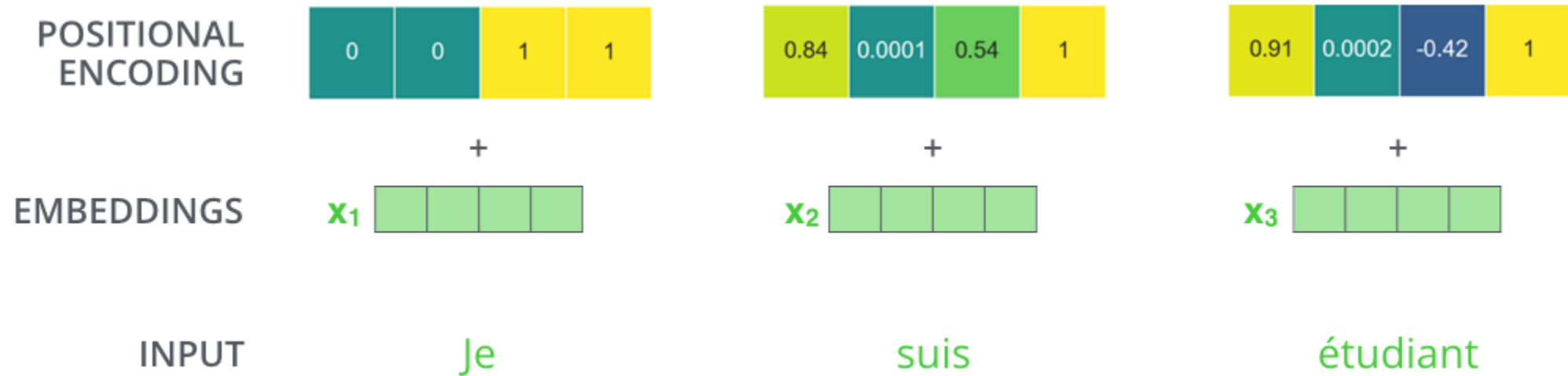
Representing The Order of The Sequence Using Positional Encoding



Positional Encoding

- Problems with absolute positions
 1. Unbounded (backpropagation in NN does not like large numbers)
 2. Gradient flow is disturbed due to discrete numbers
 3. Relative positions are not captured (it should be periodic)

Representing The Order of The Sequence Using Positional Encoding



What is positional encoding?

$$PE(pos, 2i) = \sin \frac{pos}{10000^{d_{model}/2}}$$

$$PE(pos, 2i + 1) = \cos \frac{pos}{10000^{d_{model}/2}}$$

Sentence 1

YOUR

CAT

IS

Sentence 2

LIGHT

IS

OFF

We only need to compute the positional encodings once and then reuse them for every sentence, no matter if it is training or inference.

The 10000 in the denominator is a scaling factor. It's chosen to be a large number so that the frequencies generated by the exponent are relatively small. This ensures that the positional encodings don't dominate the word embeddings



Positional Embedding

$$\begin{aligned} e'_w &= e_w + \left[\sin \left(\frac{pos}{10000^0} \right), \cos \left(\frac{pos}{10000^0} \right), \sin \left(\frac{pos}{10000^{2/4}} \right), \cos \left(\frac{pos}{10000^{2/4}} \right) \right] \\ &= e_w + \left[\sin(pos), \cos(pos), \sin \left(\frac{pos}{100} \right), \cos \left(\frac{pos}{100} \right) \right] \end{aligned}$$

$$\text{PE}(pos, 2i) = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right),$$

$$\text{PE}(pos, 2i + 1) = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right).$$

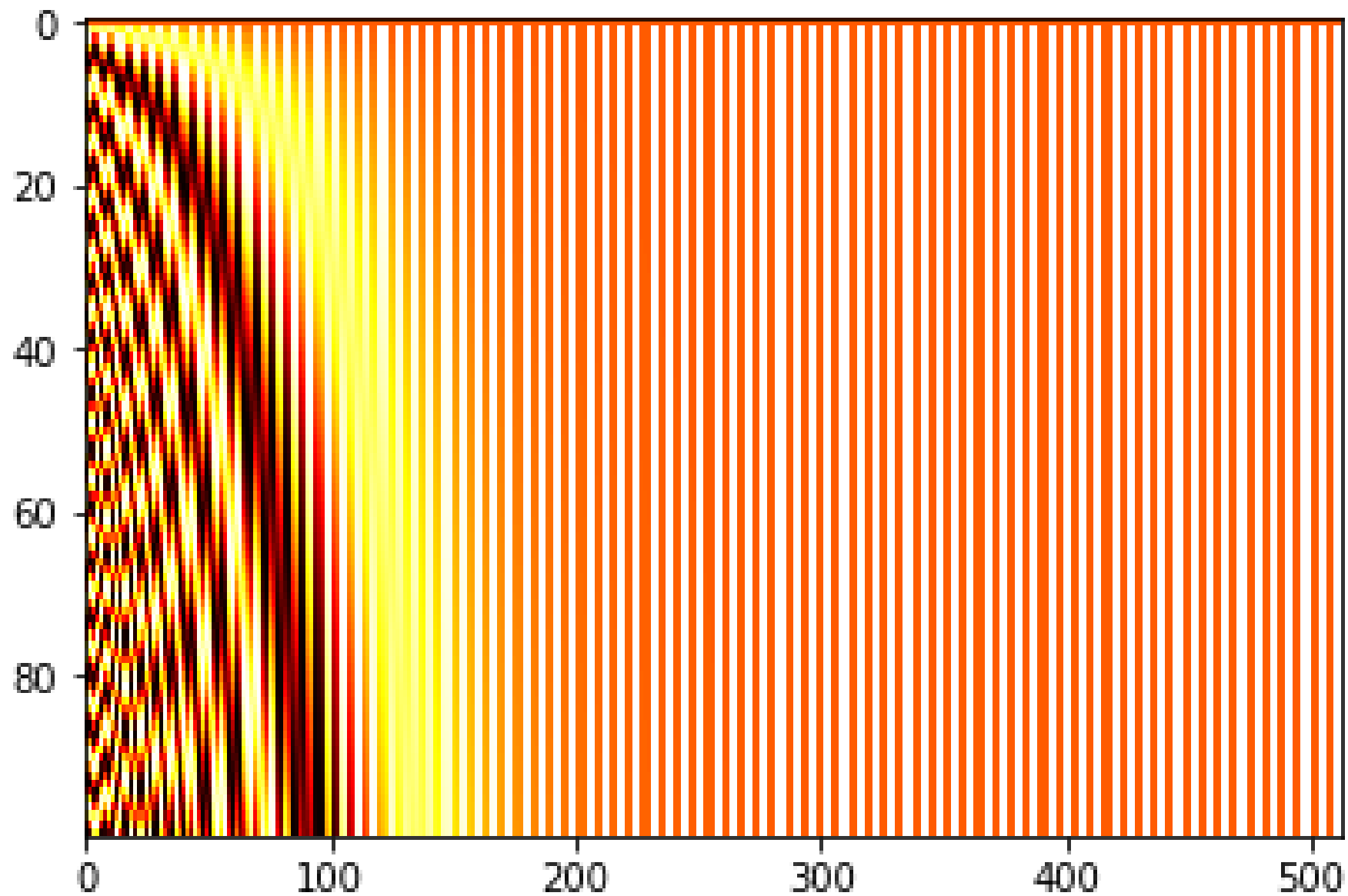
Sequence	Index of token, k	Positional Encoding Matrix with d=4, n=100			
		i=0	i=0	i=1	i=1
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

Practice Question

Given a model dimension $d_{\text{model}} = 10$, calculate the positional encoding vector for position $\text{pos} = 3$. Show all steps for computing each component of the positional vector.

Positional Embedding



Why sin cosine is used

- Because sin and cos are **analytic functions**, the model can infer the encoding for:
 - A position it never saw during training
 - A sequence longer than the training max length

Why sin cosine is used

Reason

Uniqueness

Order information

Generalization

Multi-frequency representation

Relative position inference

Parameter-free

Explanation

Every position gets a distinct vector

Smooth changes encode sequence order

Works for unseen lengths

Captures both local & global patterns

Attention can compute distances

No training required

Attention can compute distances (we need both sin and cosine for computing distances)

$\sin(a - b)$ and $\cos(a - b)$ can be computed from $\sin(a)$, $\cos(a)$, $\sin(b)$, $\cos(b)$

$$\sin(a - b) = \sin(a) \cos(b) - \cos(a) \sin(b)$$

$$\cos(a - b) = \cos(a) \cos(b) + \sin(a) \sin(b)$$

Attention can compute distances

In attention, we compute:

$$\text{score} = Q_i \cdot K_j$$

Where:

$$Q_i = W_Q (X_i + \text{PE}(i))$$

$$K_j = W_K (X_j + \text{PE}(j))$$

Because W_Q and W_K are **learned linear projections**, the dot product:

$Q_i \cdot K_j$ can learn to combine $\sin(i)$, $\cos(i)$ with $\sin(j)$, $\cos(j)$ in EXACTLY the way needed to compute functions of $i - j$

Dot product of sinusoidal encodings gives relative position

$$Q_i \cdot K_j = (W_Q (X_i + PE(i))) \cdot (W_K (X_j + PE(j)))$$

Expand:

$$Q_i \cdot K_j = (W_Q X_i) \cdot (W_K X_j) \cdot (W_Q PE(i)) \cdot (W_K PE(j))$$

The important part for **the position** is:

$$(W_Q PE(i)) \cdot (W_K PE(j))$$

This is where distance appears.

We focus on:

$$PE(i) = [\sin(\alpha i), \cos(\alpha i), \sin(\beta i), \cos(\beta i),]$$

for different frequencies $\alpha, \beta \dots$

Dot product of sinusoidal encodings gives relative position

Consider a single frequency for simplicity:

$$PE(i) = [\sin(i), \cos(i)]$$

$$PE(j) = [\sin(j), \cos(j)]$$

Compute their dot product (ignore W_Q , W_K for a moment):

$$PE(i) \cdot PE(j) = \sin(i) \sin(j) + \cos(i) \cos(j)$$

Use the trigonometric identity:

$$\sin(i) \sin(j) + \cos(i) \cos(j) = \cos(i - j)$$

The dot product equals $\cos(i - j)$!

This is the key result.

Attention can compute distances

The model naturally learns:

attention score $\approx$ some function of $(i - j)$

This is why sinusoidal positional encodings allow Transformers to *implicitly* handle:

- Relative position
- Word order
- Distance-based attention preferences

Attention can compute distances

Attention can compute distances because sinusoidal encodings allow relative position differences ($i - j$) to be reconstructed using linear combinations inside the QK^T dot product.

This gives the model a mathematically clean way to understand:

- Word order
- How far words are apart
- Directional relationships
- Long-range and short-range dependencies

The model learns the trend, not the number

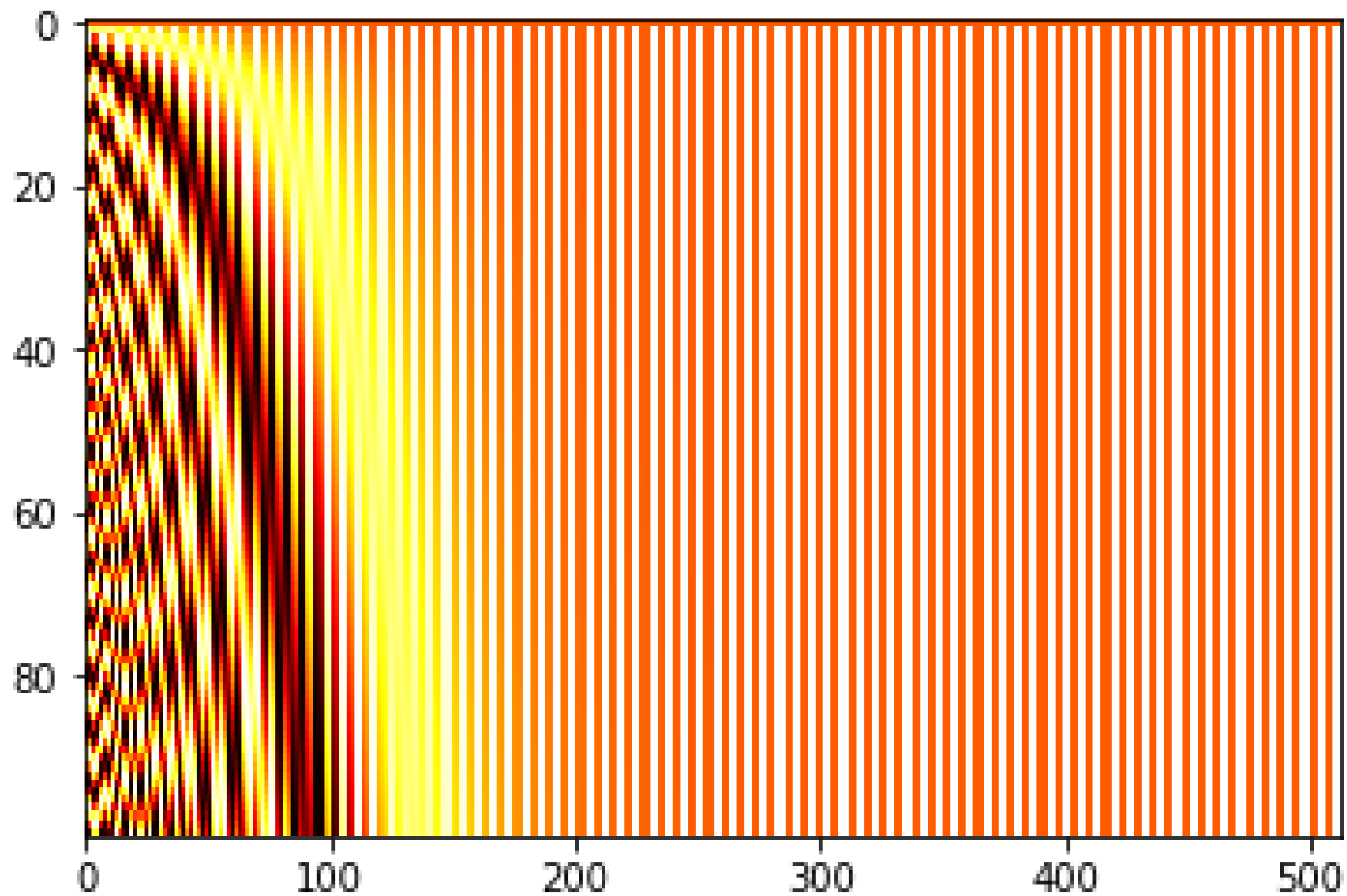
The Transformer doesn't care about the actual value of $\sin(\text{pos})$.
It cares about **how the value changes** as pos increases.

Think of it like reading a slope:

- Increasing $\rightarrow$ moving forward
- Decreasing $\rightarrow$ moving backward
- Amount of change $\rightarrow$ how far apart positions are

This works only because $\sin/\cos$ are **smooth, continuous functions**.

Positional Embedding



How Transformers Encode Local and Long-Range Word Order

- High frequency → changes fast → captures local order
- Low frequency → changes slowly → captures long-range order
- Some dimensions **change slowly** across positions → capture long-range/global position
- Some dimensions **change rapidly** → capture local/fine position differences

High frequency $\rightarrow$ changes fast

Example high-frequency wave:

$\sin(\text{pos} / 1)$

Values change a lot when the position moves by even 1 step:

$\text{pos} = 5 \rightarrow \sin(5) \approx -0.96$

$\text{pos} = 6 \rightarrow \sin(6) \approx -0.28$

$\text{pos} = 7 \rightarrow \sin(7) \approx 0.66$

A small move in sequence $\rightarrow$ big change in encoding.

Low frequency $\rightarrow$ changes slowly

A low-frequency wave looks like:

$\sin(\text{pos} / 1000)$

Values change **very slowly**:

$\text{pos} = 5 \rightarrow \sin(0.005) \approx 0.005$

$\text{pos} = 6 \rightarrow \sin(0.006) \approx 0.006$

$\text{pos} = 100 \rightarrow \sin(0.1) \approx 0.0998$

$\text{pos} = 500 \rightarrow \sin(0.5) \approx 0.4794$

$\text{pos} = 1000 \rightarrow \sin(1.0) \approx 0.8400$

Many positions share nearly the same value.

How Transformers Encode Local and Long-Range Word Order

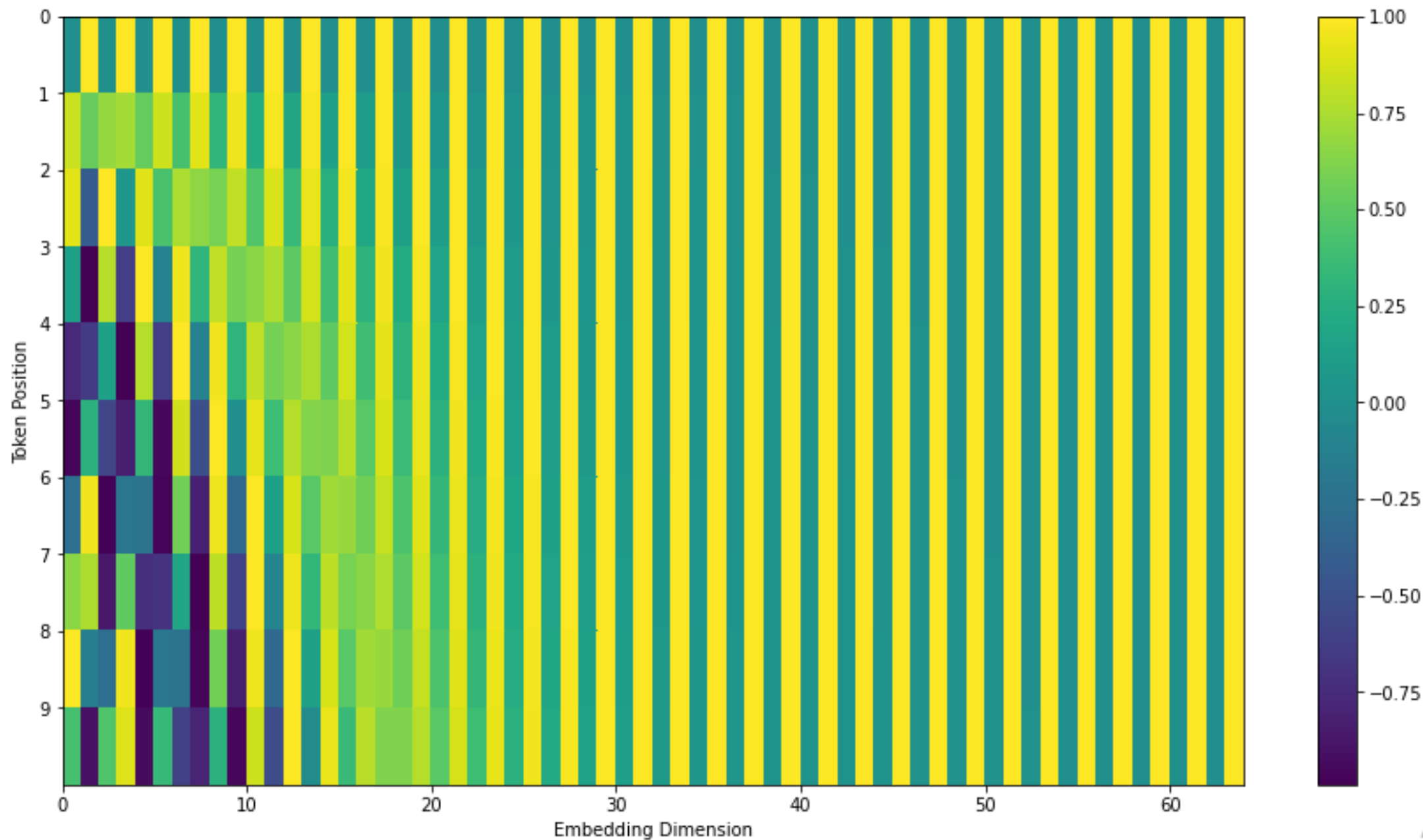
So high-frequency dimensions give strong signals for:

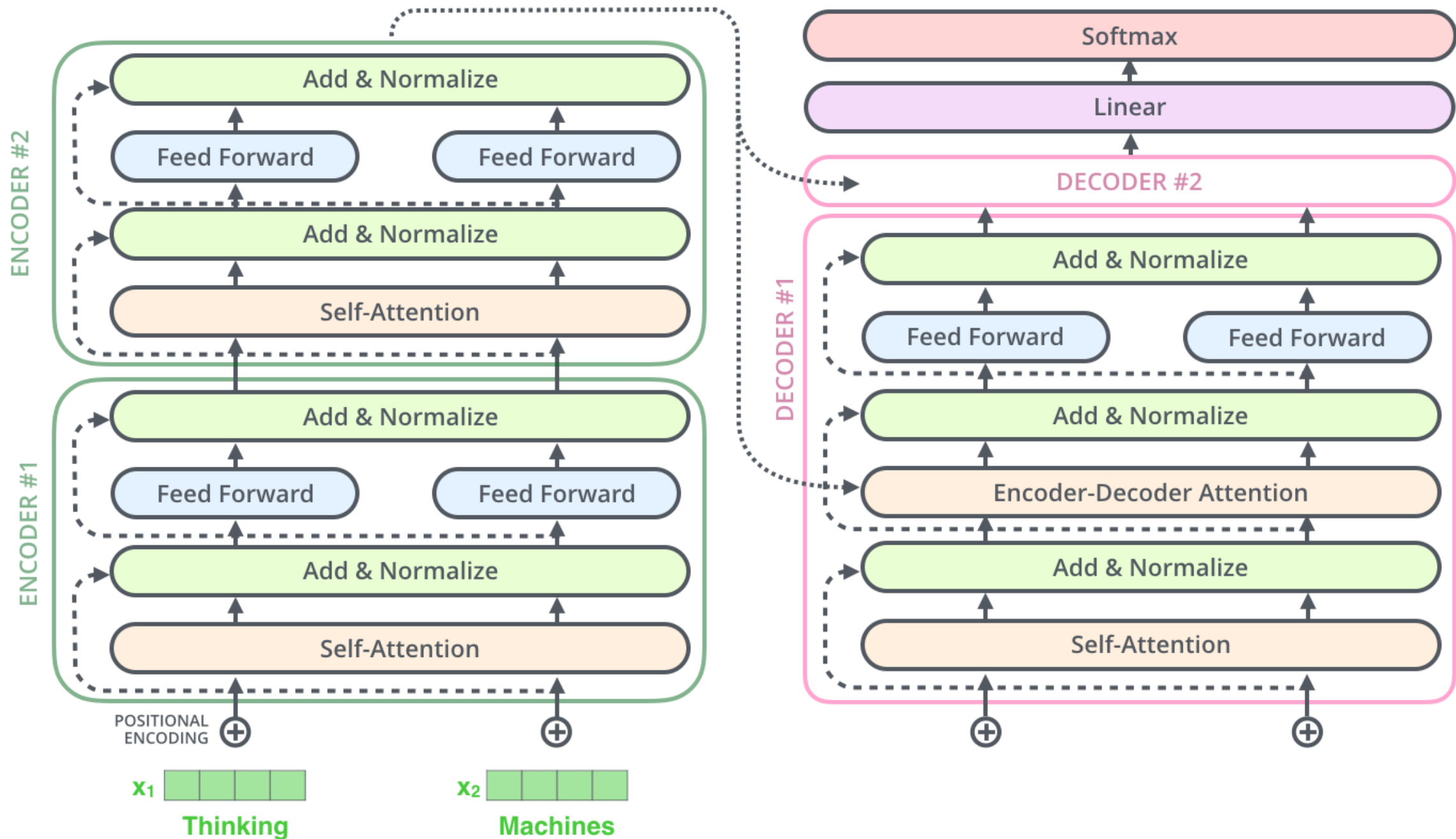
- immediate neighbors
- “order of words”

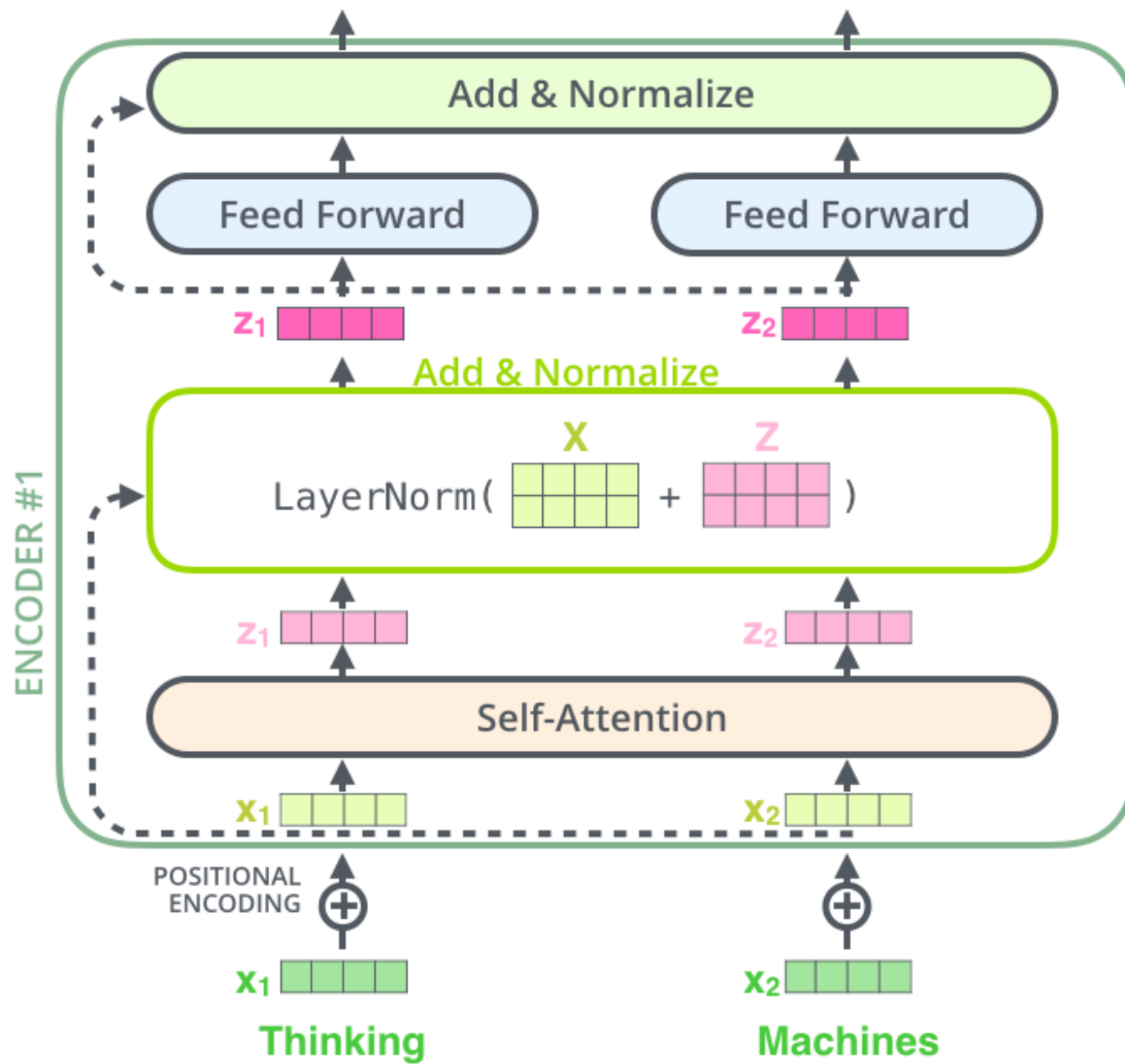
Low-frequency dimensions give signals for:

- far-apart words
- sentence-level position
- structural alignment

Positional Embedding



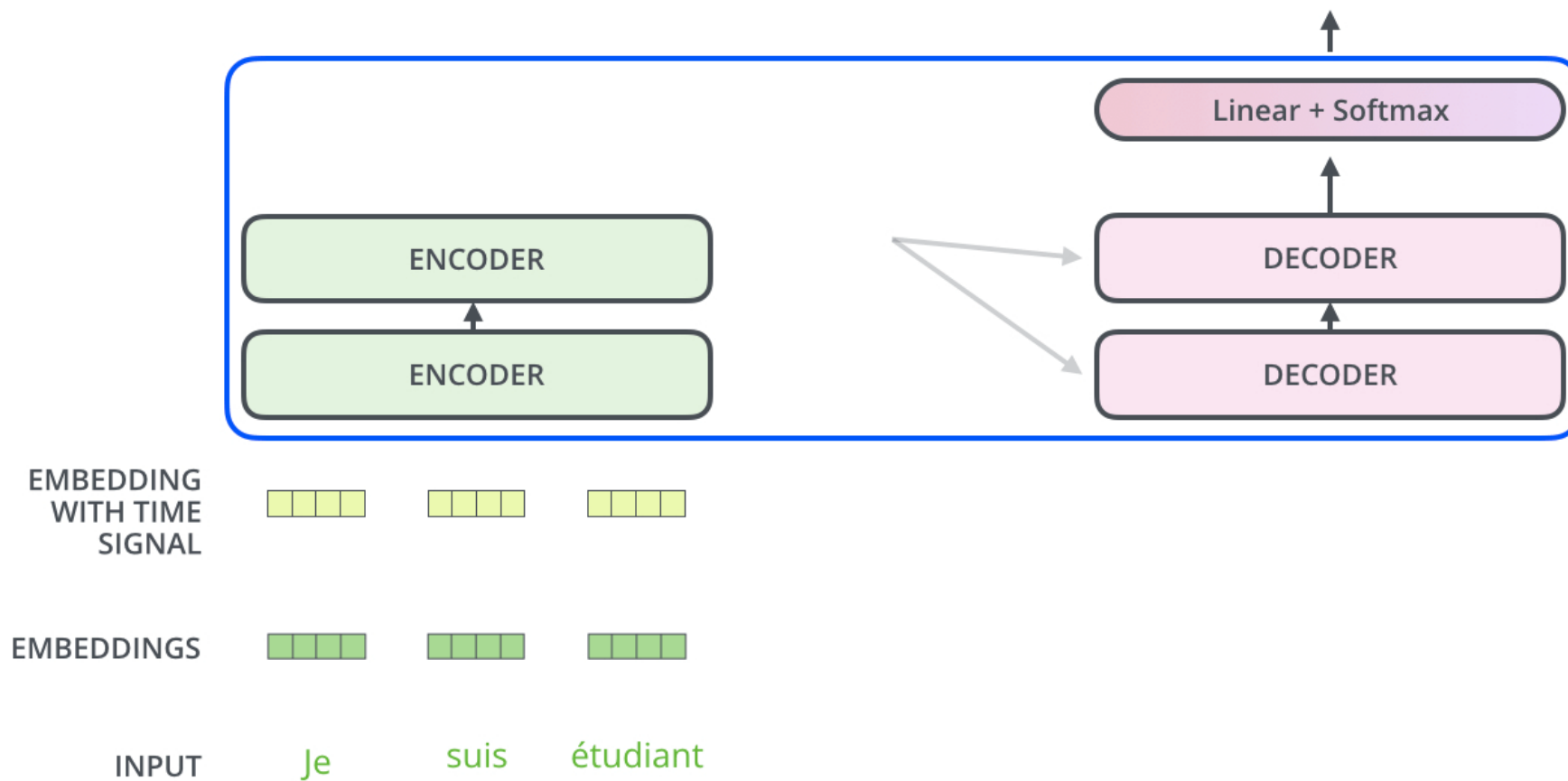




The Decoder Side

Decoding time step: 1 2 3 4 5 6

OUTPUT



Attention in Decoder

- The self attention layers in the decoder operate in a slightly different way than the one in the encoder
- In the decoder, the self-attention layer is only allowed to attend to earlier positions in the output sequence. This is done by masking future positions (setting them to $-\infty$) before the softmax step in the self-attention calculation
- The “Encoder-Decoder Attention” layer works just like multiheaded self-attention, except it creates its Queries matrix from the layer below it, and takes the Keys and Values matrix from the output of the encoder stack

Decoder Self-attention Practice Question

Consider a Transformer model translating the English sentence: " birds can fly high"

The decoder is generating the Urdu word after پرندے

Compute the decoder self-attention output for word پرندے using the provided values. $x_{\text{پرندے}}$ is the embedding for the word پرندے

$$x_{\text{پرندے}} = [0.3 \quad 2], \quad x_{\text{اونچا}} = [0 \quad 1], \quad x_{\text{اڑ}} = [3.4 \quad 2.5],$$

$$x_{\text{سکتے}} = [3 \quad 1], \quad x_{\text{ہیں}} = [1 \quad 1.6],$$

Decoder weight matrices:

$$W_Q = \begin{bmatrix} 0 & 0.7 \\ 2.5 & 1.6 \end{bmatrix}, \quad W_K = \begin{bmatrix} 0.8 & 0.7 \\ 0 & 0.5 \end{bmatrix}, \quad W_V = \begin{bmatrix} 2 & 0.5 \\ 1.5 & 1 \end{bmatrix}$$

Decoder Cross attention Practice Question

Consider a Transformer model translating the English sentence: " birds can fly high"

Urdu, so far decoded: پرندے

The decoder is generating the next Urdu word after پرندے

Compute the decoder cross-attention output for the next word after پرندے using the provided values. $d_{\text{پرندے}}$ is the decoder self-attention output for the word پرندے

$$d_{\text{پرندے}} = \begin{bmatrix} 0.3 & 2 \end{bmatrix}, \quad W_Q = \begin{bmatrix} 0 & 0.7 \\ 2.5 & 1.6 \end{bmatrix}$$

Encoder keys and values:

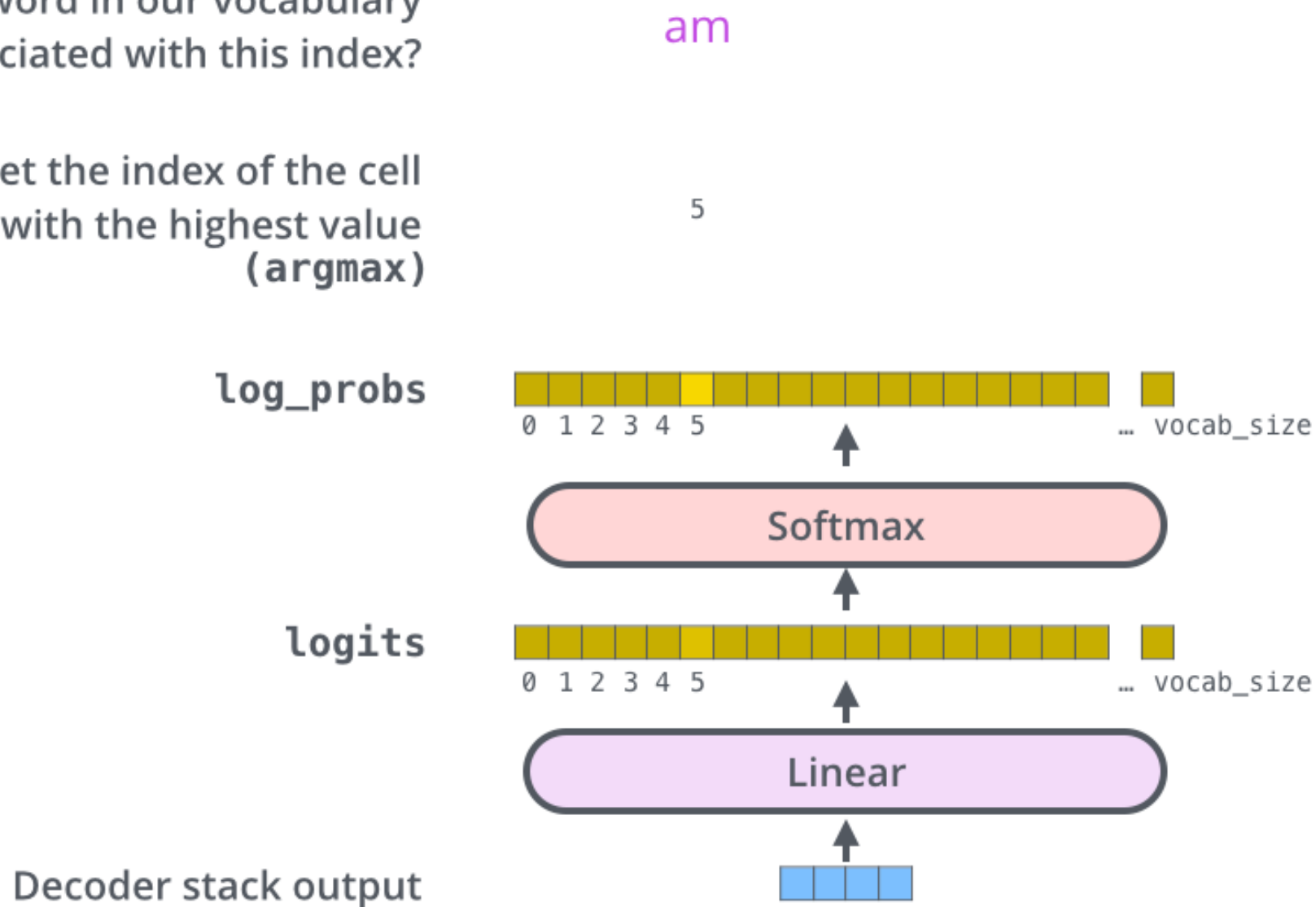
$$K_{\text{birds}} = \begin{bmatrix} 1 & 0 \end{bmatrix}, K_{\text{can}} = \begin{bmatrix} 1.5 & 0.5 \end{bmatrix}, K_{\text{fly}} = \begin{bmatrix} 2 & 1 \end{bmatrix}, K_{\text{high}} = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

$$V_{\text{birds}} = \begin{bmatrix} 3 & 1 \end{bmatrix}, V_{\text{can}} = \begin{bmatrix} 1 & 0.5 \end{bmatrix}, V_{\text{fly}} = \begin{bmatrix} 4 & 1.5 \end{bmatrix}, V_{\text{high}} = \begin{bmatrix} 0 & 2 \end{bmatrix}$$

The Final Linear and Softmax Layer

Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(**argmax**)



Recap Of Training

Output Vocabulary

WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

One-hot encoding of the word "am"

0.0	1.0	0.0	0.0	0.0	0.0
-----	-----	-----	-----	-----	-----

Untrained Model Output



Correct and desired output



a

am

I

thanks

student

<eos>

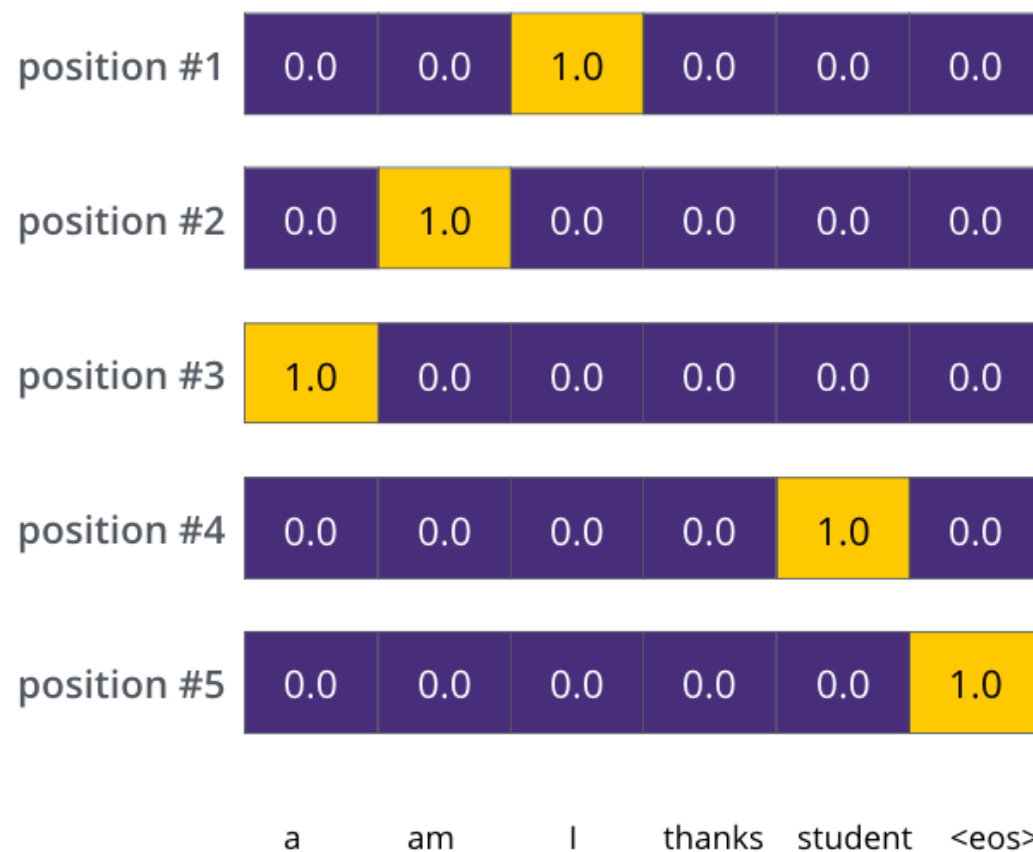


The Loss Function

- Each probability distribution is represented by a vector of width `vocab_size` (6 in our toy example, but more realistically a number like 3,000 or 10,000)
- The first probability distribution has the highest probability at the cell associated with the word “i”
- The second probability distribution has the highest probability at the cell associated with the word “am”
- And so on, until the fifth output distribution indicates

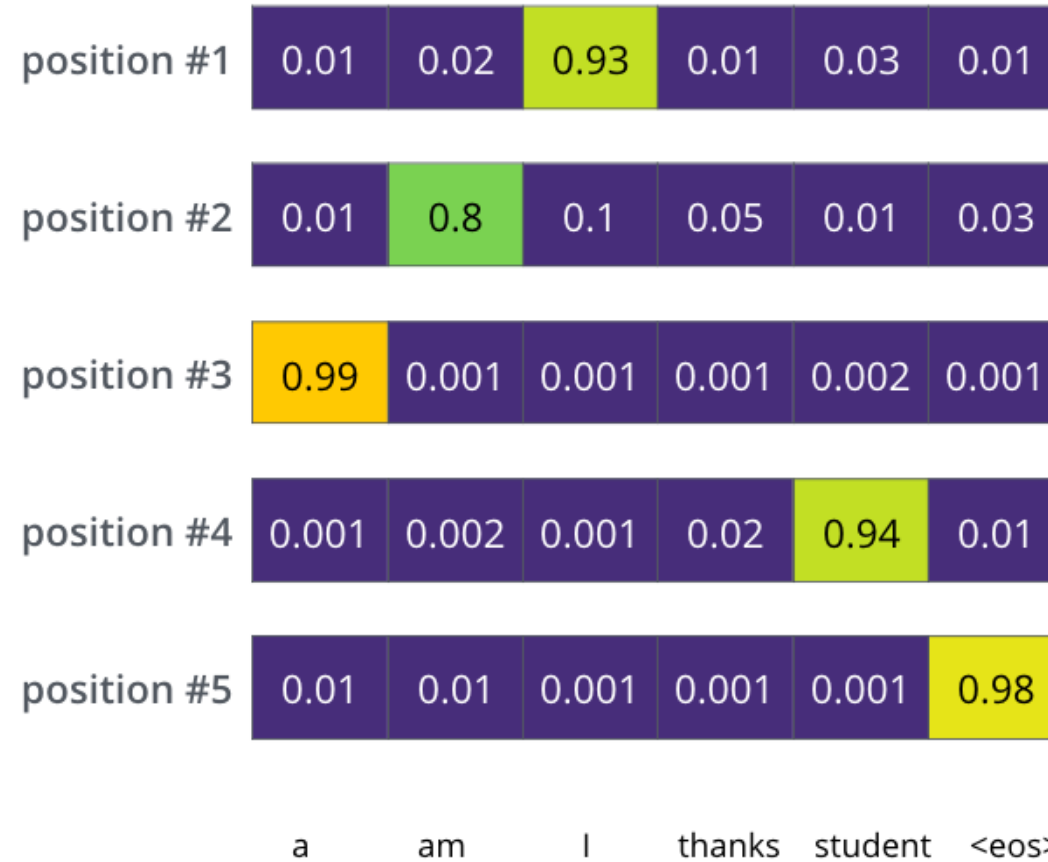
Target Model Outputs

Output Vocabulary: a am I thanks student <eos>



Trained Model Outputs

Output Vocabulary: a am I thanks student <eos>



Beam Search



Cross-entropy:

$$\text{CE}_t = - \sum_i y_{t,i} \log p_{t,i} = - \log p_{t,\text{correct}}$$

Total loss:

$$\mathcal{L} = \sum_{t=1}^T \text{CE}_t$$

Cross Entropy loss Question for Transformer

Consider a Transformer model translating the English sentence: " birds can fly high"

”پرندے اونچا اڑ سکتے ہیں”

One hot encoded vector of words is given as follows:

پرندے = [0 0 1 0 0 0 0 0]

اونچا = [0 0 0 0 0 0 0 1]

اڑ = [0 0 0 0 1 0 0 0]

سکتے = [0 0 0 0 0 0 1 0]

ہیں = [0 0 0 1 0 0 0 0]

The decoder begins with a special <SOS> token as its initial input, which signals the start of the sentence to the model.

The predicted output distribution of words at different time stamps is as follows:

$y_1 = [0.3 \ 0.1 \ 0.4 \ 0.2 \ 0 \ 0 \ 0 \ 0]$

$y_2 = [0.1 \ 0.2 \ 0.4 \ 0 \ 0.1 \ 0 \ 0 \ 0.2]$

$y_3 = [0.1 \ 0.1 \ 0.2 \ 0.4 \ 0.2 \ 0 \ 0 \ 0]$

$y_4 = [0 \ 0.1 \ 0.4 \ 0.2 \ 0 \ 0 \ 0.3 \ 0]$

$y_5 = [0.1 \ 0 \ 0.4 \ 0.3 \ 0.2 \ 0 \ 0 \ 0]$

Compute the cross-entropy loss for this sentence.

Solution for Encoder Decoder

Pick the probability corresponding to the correct index:

$y_1 \rightarrow$ پرندے index 3 $\rightarrow 0.4$

$y_2 \rightarrow$ اونچا index 8 $\rightarrow 0.2$

$y_3 \rightarrow$ اڑ index 5 $\rightarrow 0.2$

$y_4 \rightarrow$ سکتے index 7 $\rightarrow 0.3$

$y_5 \rightarrow$ ہیں index 4 $\rightarrow 0.3$

$$= - 1/5[\log(0.4) + \log(0.2) + \log(0.2) + \log(0.3) + \log(0.3)]$$

References

- [Jay Alammar](https://jalammar.github.io/illustrated-transformer/) <https://jalammar.github.io/illustrated-transformer/>
- <https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/>
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. **Attention is all you need**. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*,